

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
ИНСТИТУТ ПЕРСПЕКТИВНОЙ ИНЖЕНЕРИИ
ДЕПАРТАМЕНТ ЦИФРОВЫХ, РОБОТЕХНИЧЕСКИХ
СИСТЕМ И ЭЛЕКТРОНИКИ

КУРСОВАЯ РАБОТА
по дисциплине
«ИНТЕРНЕТ ТЕХНОЛОГИИ»
на тему:
«Разработка веб-приложения для мониторинга здоровья»

Выполнила:
Колган Мария Ивановна
Студентка 4 курса группы ПИН-б-з-22-1
Направления подготовки
09.03.03 Прикладная информатика
заочной формы обучения

(подпись)
Руководитель работы:
Щеголев А.А.
(ФИО)

Работа допущена к защите _____
(подпись руководителя) (дата)

Работа выполнена и
защищена с оценкой _____ Дата защиты _____

Члены комиссии:

_____ (должность)	_____ (подпись)	_____ (И.О. Фамилия)
_____	_____	_____
_____	_____	_____
_____	_____	_____

Ставрополь, 2026 г

УТВЕРЖДАЮ

директор департамента цифровых,
робототехнических систем
и электроники
_____Азаров И.В.

Институт Перспективной инженерии
Департамент Цифровых, робототехнических систем и электроники
Направление подготовки 09.03.03
Направленность (профиль) Прикладная информатика

ЗАДАНИЕ
на курсовую работу/проект

студентки Колган Мария Ивановна
(фамилия, имя, отчество)

по дисциплине Интернет-технологии

1. Тема работы «Разработка веб-приложения для мониторинга здоровья»

2. Цель - разработать веб-приложение для мониторинга здоровья, обеспечивающее регистрацию, хранение и наглядное представление ключевых показателей здоровья пользователя (физическая активность, пульс, давление, масса тела и др.)

3. Задачи Сбор и анализ информации: Поиск и изучение литературы, статей, научных исследований и других источников информации по выбранной теме. Планирование работы: Составление плана курсовой работы, определение основных этапов и сроков выполнения. Написание текста: Оформление результатов исследования в виде структурированного текста, включающего введение, основную часть и заключение. Проведение экспериментов и практическая реализация: Если это необходимо, выполнение практических заданий, связанных с разработкой программного обеспечения, созданием веб-сайтов или проведением экспериментов. Оформление работы: Соблюдение требований к оформлению курсовой работы, включая правильное оформление списка литературы, таблиц, рисунков и схем. Представление и защита работы: Подготовка презентации и выступление перед комиссией, где студент демонстрирует результаты своего труда и отвечает на вопросы.

4. Перечень подлежащих разработке вопросов:

а) по теоретической части Исторический обзор развития интернет-технологий (в зависимости от выбранной темы: стека технологий). Основные

понятия и определения. Описание выбранного стека технологий. Анализ существующих решений и подходов.

б) по практической части Постановка задачи. Выбор методов и инструментов для решения задачи. Описание процесса разработки. Результаты эксперимента или практической реализации. Оценка эффективности предложенных решений.

5. Исходные данные:

а) по литературным источникам _____

б) по вариантам, разработанным преподавателем _____

в) иное _____

6. Список рекомендуемой литературы _____

7. Контрольные сроки представления отдельных разделов курсовой работы:

25 % - _____ “ ” _____ 20_ г.

50 % - _____ “ ” _____ 20_ г.

75 % - _____ “ ” _____ 20_ г.

100 % - _____ “ ” _____ 20_ г.

8. Срок защиты студентом курсовой работы “ ” _____ 20_ г.

Дата выдачи задания “ ” _____ 20_ г.

Руководитель курсовой работы

Старший преподаватель департамента цифровых, робототехнических систем и электроники института перспективной инженерии Щеголев А.А.

(ученая степень, звание)(личная подпись)(инициалы, фамилия)

Задание приняла к исполнению студентка заочной формы обучения 4 курса ПИН-б-3-22-1 группы _____

(личная подпись)

(инициалы, фамилия)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ВЕБ-ПРИЛОЖЕНИЙ.....	8
1.1 Исторический обзор развития интернет-технологий.....	8
1.2 Основные понятия веб-разработки.....	9
1.3. Описание выбранного стека технологий	10
1.4 Анализ существующих решений для мониторинга здоровья.....	11
2 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ МОНИТОРИНГА ЗДОРОВЬЯ.....	12
2.1 Постановка задачи.....	12
2.2 Выбор средства создания веб-приложения.....	13
2.3 Описание объектной модели.....	14
2.4 Генерация базы данных, разработка серверной части.....	14
2.5 Разработка клиентской части и вёрстка.....	15
2.6 Использование системы контроля версий, развертывание.....	16
2.7 Тестирование и верификация работоспособности.....	17
ЗАКЛЮЧЕНИЕ	18
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ.....	20
ПРИЛОЖЕНИЕ	24

ВВЕДЕНИЕ

Актуальность темы. В последние годы наблюдается устойчивый рост интереса к вопросам личного здоровья и здорового образа жизни. Современные люди всё чаще используют цифровые инструменты для отслеживания физической активности, контроля питания, измерения давления, пульса, уровня стресса и других показателей. Этому способствует широкое распространение носимых устройств (фитнес-браслетов, умных часов), а также доступность мобильных приложений и веб-сервисов в сфере e-Health.

Однако многие существующие решения имеют ограничения: они либо привязаны к конкретным производителям устройств, либо требуют установки приложений на смартфон, либо не предоставляют удобного веб-интерфейса для анализа долгосрочной статистики. Кроме того, зачастую отсутствует возможность централизованного хранения данных, полученных от разных источников, и их совместного анализа. В связи с этим разработка веб-приложения для мониторинга здоровья, которое обеспечивало бы удобный сбор, хранение, визуализацию и анализ показателей здоровья, является актуальной задачей, востребованной как частными пользователями, так и медицинскими учреждениями при дистанционном наблюдении за пациентами.

Объект исследования – процесс сбора, обработки и визуализации данных о состоянии здоровья пользователя с использованием интернет-технологий.

Предмет исследования – методы и инструменты разработки веб-приложения для мониторинга здоровья, включая проектирование пользовательского интерфейса, организацию хранения данных и реализацию серверной логики.

Цель работы – разработать веб-приложение для мониторинга здоровья, обеспечивающее регистрацию, хранение и наглядное представление ключевых показателей здоровья пользователя (физическая активность, пульс, давление, масса тела и др.), а также предоставляющее возможность отслеживания динамики показателей во времени.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ существующих веб-приложений и сервисов в области мониторинга здоровья, выявить их функциональные возможности и недостатки;
2. Определить набор ключевых показателей здоровья, подлежащих мониторингу, и требования к разрабатываемому приложению;
3. Выбрать современный стек технологий для реализации фронтенд- и бэкенд-частей приложения, обосновать выбор;
4. Спроектировать архитектуру веб-приложения, включая базу данных, серверную логику и пользовательский интерфейс;
5. Реализовать основные функции: регистрацию и аутентификацию пользователей, добавление и редактирование записей о показателях здоровья, визуализацию данных в виде графиков и таблиц;
6. Провести тестирование разработанного приложения и оценить его работоспособность, удобство интерфейса и производительность.

Методы исследования. При выполнении работы используются методы системного анализа, объектно-ориентированного проектирования, моделирования баз данных, а также методы веб-разработки (языки HTML, CSS, JavaScript, серверный язык программирования, система управления базами данных). Для управления версиями и совместной разработки применяется система Git.

Краткий обзор структуры работы. Курсовая работа состоит из введения, двух глав (теоретической и практической), заключения, списка литературы и приложений.

В первой главе рассматриваются теоретические аспекты мониторинга здоровья с использованием интернет-технологий: анализируются существующие аналоги, определяются основные показатели здоровья и требования к приложению, обосновывается выбор технологического стека (фронтенд, бэкенд, база данных).

Во второй главе описывается практическая реализация веб-приложения: приводится архитектура системы, схема базы данных, реализация ключевых модулей (аутентификация, ввод и редактирование данных, визуализация графиков), результаты тестирования и оценка эффективности.

В заключении подводятся итоги работы, формулируются основные выводы и намечаются направления для дальнейшего развития приложения (например, интеграция с API фитнес-трекеров, внедрение алгоритмов прогнозирования состояния здоровья).

Работа содержит список использованных источников, а также приложения с примерами экранных форм и фрагментами исходного кода.

1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ВЕБ-ПРИЛОЖЕНИЙ

1.1 Исторический обзор развития интернет-технологий

Интернет-технологии прошли длинный путь от простых текстовых страниц до сложных веб-приложений с интерактивным интерфейсом и распределённой архитектурой. В 1990-е годы веб-сайты были статическими, построенными на HTML и CSS. С появлением динамических языков (Perl, PHP, ASP) стало возможным генерировать страницы на сервере. В начале 2000-х технологии AJAX позволили обновлять части страницы без перезагрузки, что положило начало эре «Web 2.0» и одностраничным приложениям (SPA).

В последние годы активно развиваются архитектуры микросервисов, контейнеризация (Docker), облачные платформы и инструменты автоматизации CI/CD. Фреймворки для фронтенда (React, Vue.js, Angular) и бэкенда (Node.js, Django, FastAPI) стали стандартом индустрии. Всё это позволяет создавать масштабируемые, безопасные и удобные веб-приложения, что особенно важно для таких областей, как телемедицина и мониторинг здоровья.

1.2 Основные понятия веб-разработки

Веб-приложение – это клиент-серверная программа, в которой клиентом выступает браузер, а серверная часть обеспечивает хранение данных, бизнес-логику и взаимодействие с базами данных. Основные компоненты:

- Фронтенд (front-end) – пользовательский интерфейс (формы ввода, графики, таблицы), реализуемый с помощью HTML, CSS и JavaScript. Обычно используется фреймворк (например, Vue.js) для управления состоянием и рендерингом;
- Бэкенд (back-end) – серверная логика (аутентификация, обработка запросов, сохранение показателей), написанная на одном из языков программирования (Python, JavaScript, Java, C# и др.). Современные бэкенд-фреймворки (FastAPI, Express.js) позволяют быстро создавать RESTful API;
- База данных (БД) – хранилище информации о пользователях, их показателях здоровья. Используются реляционные (PostgreSQL, MySQL) или NoSQL (MongoDB) системы;
- API – программный интерфейс, через который фронтенд общается с бэкендом. Обычно используется REST или GraphQL;
- Контейнеризация и оркестрация – Docker и Docker Compose изолируют сервисы и упрощают развёртывание;
- Система контроля версий (Git) – позволяет отслеживать изменения в коде, работать в команде и автоматизировать развёртывание (CI/CD).

1.3. Описание выбранного стека технологий

Для реализации веб-приложения «Мониторинг здоровья» был выбран современный стек, обеспечивающий высокую производительность, безопасность и удобство разработки. Основные компоненты приведены в таблице 1.1.

Обоснование выбора каждого инструмента:

1. Python – язык с простым синтаксисом, что сокращает время разработки; множество библиотек для веб-разработки и работы с БД.

2. FastAPI – один из самых быстрых Python-фреймворков (асинхронный), автоматически проверяет типы данных, генерирует интерактивную документацию, что упрощает отладку.

3. PostgreSQL – стандарт в промышленной разработке: поддерживает сложные запросы, внешние ключи, транзакции. Подходит для хранения структурированных медицинских показателей.

4. JWT – позволяет не хранить сессии на сервере, масштабируется горизонтально; пара токенов (короткий access + длинный refresh) повышает безопасность.

5. Vue.js – легковесный, интуитивно понятный, хорошо интегрируется с Pinia для управления состоянием. Позволяет создавать отзывчивый интерфейс без перезагрузки страниц.

6. Docker – гарантирует, что приложение будет работать одинаково на компьютере разработчика, на сервере тестирования и на боевом сервере. Каждый сервис изолирован в своём контейнере.

7. GitHub Actions – автоматизирует процесс: при каждом изменении запускаются проверки, а при слиянии с основной веткой происходит автоматический деплой на сервер.

1.4 Анализ существующих решений для мониторинга здоровья

На рынке представлено множество решений. Краткий обзор наиболее популярных приведён в таблице 1.2.

Анализ показывает, что многие существующие системы либо привязаны к конкретным устройствам, либо не предоставляют удобного веб-доступа, либо не позволяют пользователю гибко настраивать набор отслеживаемых показателей. Это определяет актуальность разработки собственного веб-приложения, свободного от указанных недостатков.

2 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ МОНИТОРИНГА ЗДОРОВЬЯ

2.1 Постановка задачи

В рамках практической части разработано веб-приложение, которое позволяет пользователю:

1. Зарегистрироваться и войти в систему (аутентификация через JWT);
2. Управлять профилем (ФИО, пол, дата рождения, рост);
3. Добавлять, просматривать и удалять измерения по следующим показателям: артериальное давление (верхнее/нижнее), пульс, вес, температура тела, сахар в крови, SpO₂, количество шагов;
4. Просматривать историю каждого показателя в виде таблицы и интерактивного графика (линейная диаграмма);
5. Ставить цели (например, «снизить вес до 70 кг к 01.10.2025») и отслеживать прогресс;
6. Видеть на дашборде последние значения показателей и активные цели.

Приложение должно быть реализовано как Single Page Application (SPA) с отдельными бэкендом и базой данных. Развёртывание должно быть автоматизировано с использованием Docker и GitHub Actions.

2.2 Выбор средства создания веб-приложения

В качестве средств разработки выбраны:

- Бэкенд: Python + FastAPI – обеспечивает высокую производительность и асинхронную обработку запросов;
- Фронтенд: Vue.js (Composition API) – интуитивно понятный, реактивный, хорошо подходит для создания SPA;
- База данных: PostgreSQL – надёжная реляционная СУБД, поддерживающая сложные запросы и транзакции;
- ORM и миграции: SQLAlchemy + Alembic – позволяют работать с БД через Python-объекты и управлять схемой;
- Контейнеризация: Docker и Docker Compose – обеспечивают изоляцию и воспроизводимость окружения;
- CI/CD: GitHub Actions – автоматизация сборки, тестирования и деплоя.

Обоснование выбора подробно дано в разделе 1.3.

2.3 Описание объектной модели

Объектная модель отражает структуру данных приложения. Основные сущности и их взаимосвязи представлены на диаграмме классов в таблице 2.1.

2.4 Генерация базы данных, разработка серверной части

Генерация базы данных производится с помощью миграций Alembic. Первая миграция создаёт все перечисленные выше таблицы. Для наполнения тестовыми данными разработан скрипт-сид (seeder), который генерирует случайные измерения для трёх вымышленных пользователей за период с января по июнь 2026 года. Пример фрагмента кода сйда:

Скрипт использует параметры base (базовое значение), trend (тренд), std (стандартное отклонение) и freq (частота измерений) для каждого показателя, что позволяет имитировать реалистичные данные.

Разработка серверной части (API) выполнена на FastAPI. Все эндпоинты сгруппированы по тематическим модулям (таблица 2.2).

Аутентификация реализована через JWT. Каждый защищённый эндпоинт проверяет наличие валидного access-токена в заголовке Authorization: Bearer <token>. Пользователь из токена используется для фильтрации данных – никто не может получить или изменить чужие записи. Пароли хранятся в хешированном виде (bcrypt).

2.5 Разработка клиентской части и вёрстка

Фронтенд разработан на Vue.js (Composition API) с использованием Pinia для глобального состояния. Структура страниц:

- Dashboard.vue – главная страница: карточки с последними значениями каждого показателя, прогресс по активным целям.
- Measurements.vue – выбор типа показателя, таблица с историей, форма добавления нового измерения, график ApexCharts.
- Goals.vue – список целей, форма создания новой цели, прогресс-бары.
- Profile.vue – редактирование личных данных.

Для стилизации использованы Tailwind CSS (утилитарные классы) и компоненты PrimeVue (модальные окна, кнопки, поля ввода). Графики строятся библиотекой ApexCharts – они интерактивны, поддерживают масштабирование и отображение значений при наведении.

Автоматическое обновление access-токена реализовано через перехватчик (axios interceptor): при получении ошибки 401 клиент пытается отправить refresh-токен на эндпоинт /refresh, и если успешно – повторяет исходный запрос.

Вёрстка адаптивна, корректно отображается на экранах разных размеров (десктоп, планшет, смартфон).

2.6 Использование системы контроля версий, развертывание

Система контроля версий Git используется на протяжении всей разработки. Репозиторий размещён на GitHub. Ведётся работа с ветками: main – стабильная версия, develop – разработка, отдельные ветки для новых функций. Каждое изменение сопровождается осмысленным сообщением коммита.

Развертывание автоматизировано через Docker и GitHub Actions. Приложение упаковано в контейнеры:

- db – PostgreSQL с именованным томом для хранения данных.
- backend – сборка из backend/Dockerfile, запуск с использованием gunicorn и uvicorn.
- frontend – сборка статических файлов через Vite, затем запуск Nginx, который их раздаёт и проксирует API.

Настроен CI/CD пайплайн (файл .github/workflows/ci.yml):

- При создании Pull Request – запускаются линтеры и сборка для проверки работоспособности.
- При пуше в ветку main – собираются Docker-образы, публикуются в GitHub Container Registry (GHCR), затем по SSH на сервере скачиваются новые образы и перезапускаются контейнеры.

Таким образом, обновление приложения на сервере происходит полностью автоматически, без участия разработчика.

2.7 Тестирование и верификация работоспособности

Проведено функциональное тестирование:

- Регистрация – успешное создание пользователя, ошибка при попытке зарегистрировать существующий email.
- Аутентификация – вход с правильным паролем выдаёт токены; с неправильным – отказ.
- Добавление измерений – значения сохраняются, отображаются в истории и на графике.
- Цели – прогресс пересчитывается автоматически при добавлении новых измерений.
- Выход – токены удаляются на клиенте, дальнейшие запросы требуют повторного входа.

Также выполнено кросс-браузерное тестирование (Chrome, Firefox, Edge) и проверка адаптивной вёрстки на экранах разных размеров. Все критичные сценарии работают корректно.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы на тему «Разработка веб-приложения для мониторинга здоровья» была достигнута поставленная цель: создано полнофункциональное веб-приложение, позволяющее пользователям вести личную историю показателей здоровья, отслеживать динамику через графики и ставить цели.

В рамках работы были решены следующие задачи:

1. Проведён анализ предметной области – изучены существующие аналоги (Google Fit, Apple Health, MyFitnessPal, Fitbit Dashboard), выявлены их недостатки: привязка к конкретным устройствам, отсутствие полноценного веб-интерфейса, платные подписки для расширенной статистики. Это подтвердило актуальность разработки собственного решения.
2. Определены функциональные требования – приложение поддерживает регистрацию и аутентификацию на основе JWT, управление профилем, добавление и просмотр измерений по девяти типам, визуализацию динамики с помощью графиков, постановку целей и отслеживание прогресса.
3. Выбран и обоснован технологический стек: бэкенд на Python 3.12 с FastAPI, база данных PostgreSQL, ORM SQLAlchemy + Alembic, фронтенд на Vue.js с Pinia, стилизация Tailwind CSS + PrimeVue, контейнеризация Docker, CI/CD на GitHub Actions.
4. Спроектирована архитектура приложения – трёхзвенная клиент-серверная, где Nginx выступает обратным прокси, а база данных изолирована внутри Docker-сети.

5. Реализовано и протестировано приложение – все критичные сценарии работают корректно, интерфейс адаптирован для разных размеров экранов.
6. Обеспечен высокий уровень безопасности – пароли хешируются, данные изолированы через проверку user_id из токена, настроен CORS, секреты хранятся в GitHub Secrets.

Практическая значимость работы заключается в создании готового к использованию веб-приложения, которое может применяться любым пользователем для ведения дневника здоровья и может быть адаптировано для медицинских организаций.

Перспективы дальнейшего развития включают интеграцию с API фитнес-трекеров, внедрение алгоритмов прогнозирования, push-уведомления, разработку мобильного приложения и расширение ролей (врач-пациент).

Работа выполнена в полном соответствии с заданием, оформлена согласно требованиям методических указаний. Разработанный программный продукт может быть рекомендован к практическому использованию.

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

- 1) ГОСТ 19.101-2024. Единая система программной документации. Виды программ и программных документов. – Введ. 01.03.2025. – Москва : Российский институт стандартизации, 2024. – 12 с;
- 2) ГОСТ 19.101-77. Единая система программной документации. Виды программ и программных документов. – Взамен ГОСТ 19.101-77 ; введ. 01.01.1980. – Москва : Издательство стандартов, 1978. – 6 с;
- 3) ГОСТ 34.201-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Виды, комплектность и обозначение документов при создании автоматизированных систем. – Введ. 01.12.2021. – Москва : Российский институт стандартизации, 2021. – 28 с;
- 4) ГОСТ 34.603-92. Информационная технология. Виды испытаний автоматизированных систем. – Введ. 01.01.1993. – Москва : Издательство стандартов, 1992. – 14 с;
- 5) ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. – Введ. 01.06.2011. – Москва : Стандартинформ, 2011. – 98 с;
- 6) ГОСТ Р 56939-2024. Защита информации. Разработка безопасного программного обеспечения. Общие требования. – Введ. 01.01.2025. – Москва : Российский институт стандартизации, 2024. – 32 с;
- 7) ГОСТ Р 50922-2006. Защита информации. Основные термины и определения. – Введ. 01.01.2007. – Москва : Стандартинформ, 2006. – 30 с;
- 8) ГОСТ 7.82-2001. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание электронных ресурсов. Общие требования и правила составления. – Введ. 01.07.2002. – Москва : ИПК Издательство стандартов, 2002. – 12 с;

9) СанПиН 2.2.2/2.4.1340-03. Гигиенические требования к персональным электронно-вычислительным машинам и организации работы. – Утверждены постановлением Главного государственного санитарного врача Российской Федерации от 30 мая 2003 г. № 118. – Москва : Минздрав России, 2003. – 42 с;

10) СанПиН 2.4.2.2821-10. Санитарно-эпидемиологические требования к условиям и организации обучения в общеобразовательных учреждениях. – Утверждены постановлением Главного государственного санитарного врача Российской Федерации от 29 декабря 2010 г. № 189. – Москва : Минздрав России, 2011. – 56 с;

11) Браун, И. Веб-разработка с применением Node и Express. Полноценное использование стека JavaScript / Итан Браун; [пер. с англ. И. Пальти]. – Санкт-Петербург : Питер, 2017. – 336 с. : ил. – (Бестселлеры O'Reilly). – ISBN 978-5-496-02156-2;

12) Кантелон, М. Node.js в действии. Второе издание / М. Кантелон, А. Янг, Б. Мек; [пер. с англ. Т. Головайчук, Н. Райлих]. – Санкт-Петербург : Питер, 2018. – 432 с. – ISBN 978-5-4461-0878-7;

13) Флэнаган, Д. JavaScript. Полное руководство : справочник по самому популярному языку программирования / Д. Флэнаган ; перевод с английского и редакция Ю. Н. Артеменко. – 7-е изд. – Санкт-Петербург : Символ-Плюс, 2021. – 1088 с. – ISBN 978-5-907203-79-2;

14) Дронов, В. А. Node.js, Express, MongoDB и React. 23 урока для начинающих / В. А. Дронов. – Санкт-Петербург : БХВ-Петербург, 2024. – 416 с. – ISBN 978-5-9775-1853-6;

15) Прокопенко, А. В. Разработка веб-приложений на Node.js / А. В. Прокопенко. – Москва : ДМК Пресс, 2022. – 412 с. – ISBN 978-5-97060-951-8;

16) Янцев, В. В. Разработка web-страниц на HTML, CSS и JavaScript : учебное пособие для вузов / В. В. Янцев. – Санкт-Петербург : Лань, 2023. – 352 с. – ISBN 978-5-8114-9876-5;

17) Оберн, М. Проектирование архитектуры API / М. Оберн. – Москва : Эксмо, 2024. – 288 с. – ISBN 978-601-09-5053-5;

18) Никсон, Р. Создаем динамические веб-сайты с помощью PHP, MySQL, JavaScript, CSS и HTML5 / Р. Никсон ; перевод с английского А. Киселева. – 7-е изд. – Санкт-Петербург : Питер, 2024. – 832 с. – ISBN 978-601-12-3658-4;

19) Дыновски, Л. Стили API. Проектирование и внедрение / Л. Дыновски, М. Дулак. – Санкт-Петербург : Питер, 2023. – 416 с. – ISBN 978-601-14-1157-8;

20) Чакон, С. Pro Git / С. Чакон, Б. Штрауб. – 2-е изд. – [Б. м.] : Apress, 2014. – 458 с. – Режим доступа: <https://git-scm.com/book/ru/v2/> (дата обращения: 28.01.2026);

21) REST API Development with Node.js / F. Doglio. – [Б. м.] : Springer Nature, 2020. – 320 с. – ISBN 978-1-4842-5964-9;

22) Позевалкин, В. В. Разработка веб-приложений на основе клиентских каркасов и библиотек : учебное пособие / В. В. Позевалкин, Н. Ф. Панова. – Москва : Ай Пи Ар Медиа, 2021. – 215 с. – ISBN 978-5-4497-0945-7;

23) Официальная документация Node.js [Электронный ресурс]. – Режим доступа: <https://nodejs.org/docs/> (дата обращения: 28.03.2026). – Загл. с экрана;

24) Официальная документация Express.js [Электронный ресурс]. Режим доступа: <https://expressjs.com/ru/> (дата обращения: 28.03.2026). – Загл. с экрана;

25) Использование Fetch API // MDN Web Docs [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/ru/docs/Web/API/Fetch_API (дата обращения: 28.03.2026). – Загл. с экрана;

26) CORS middleware // Официальная документация Express.js [Электронный ресурс] – Режим доступа:

<https://expressjs.com/ru/resources/middleware/cors.html> (дата обращения: 27.01.2026). – Загл. с экрана;

27) Building REST APIs with Express.js: практическое руководство [Электронный ресурс] – Режим доступа: <https://www.freecodecamp.org/news/build-a-rest-api-with-express-js/> (дата обращения: 28.01.2026). – Загл. с экрана;

28) Репозиторий курсовой работы «Веб-приложение для ведения заметок» [Электронный ресурс] – Режим доступа: <https://github.com/mariyadymova06-star/health-app> (дата обращения: 01.02.2026). – Загл. с экрана.

ПРИЛОЖЕНИЕ

Компонент	Технология	Назначение
Язык бэкенда	Python 3.12	Высокая читаемость кода, богатая экосистема библиотек, скорость разработки
Веб-фреймворк	FastAPI	Асинхронность, автоматическая генерация документации (Swagger), встроенная валидация
База данных	PostgreSQL	Реляционная СУБД, надёжность, поддержка сложных запросов и целостности данных
ORM	SQLAlchemy (asyncpg)	Работа с БД через Python-объекты без написания сырого SQL, асинхронность
Миграции	Alembic	Управление изменениями схемы БД, версионирование
Аутентификация	JWT (access + refresh токены)	Безопасная передача данных авторизации, возможность обновления сессии
Фронтенд-фреймворк	Vue.js + Pinia	Создание динамического SPA, управление глобальным состоянием
Сборщик фронтенда	Vite	Быстрая разработка (мгновенная перезагрузка), оптимизированная сборка
Стилизация	Tailwind CSS + PrimeVue	Утилитарный CSS для быстрой вёрстки, готовые UI-компоненты

Графики	ApexCharts	Интерактивные графики с поддержкой адаптивности и тёмной темы
Веб-сервер	Nginx	Раздача статики и проксирование API-запросов на бэкенд
Контейнеризация	Docker + Docker Compose	Изоляция сервисов, воспроизводимость окружения, упрощение развёртывания
CI/CD	GitHub Actions + GHCR	Автоматическая сборка, тестирование и деплой при пуше в основную ветку

Таблица 1.1. Технологический стек проекта

Название	Основные функции	Платформа	Недостатки
Google Fit	Шаги, активность, пульс, синхронизация с Android	Мобильное приложение, веб-дашборд	Ограниченная аналитика, привязка к экосистеме Google
Apple Health	Агрегация данных от устройств Apple	iOS, macOS	Отсутствие полноценного веб-интерфейса
MyFitnessPal	Калории, питание, физическая активность	Мобильное, веб	Основной упор на питание, слабая медицинская составляющая
Fitbit Dashboard	Шаги, сон, пульс, давление	Веб, мобильное	Требуется фирменное устройство, платная

			подписка для расширенной статистики
--	--	--	--

Таблица 1.2 – Сравнительный анализ существующих веб-приложений

Класс	Атрибуты	Связи
User	id (PK), email (unique), hashed_password, created_at	1—1 с Profile, 1—N с Measurement, 1—N с Goal
Profile	id (PK), user_id (FK), name, gender, birth_date, height_cm, updated_at	принадлежит User
MeasurementType	id (PK), key (уникальный), name, unit, is_double (флаг)	1—N с Measurement, 1—N с Goal
Measurement	id (PK), user_id (FK), type_id (FK), value, secondary_value (для давления), measured_at, created_at, note	принадлежит User и MeasurementType
Goal	id (PK), user_id (FK), type_id (FK), start_value, target_value, start_date, end_date, is_active, created_at	принадлежит User и MeasurementType

Таблица 2.1 – Описание классов объектной модели

Метод	URL	Назначение
POST	/api/v1/auth/register	Регистрация нового пользователя
POST	/api/v1/auth/login	Вход, выдача access и refresh токенов
POST	/api/v1/auth/refresh	Обновление access-токена
POST	/api/v1/auth/logout	Выход (удаление токенов на клиенте)
GET	/api/v1/profile	Получение данных профиля
PUT	/api/v1/profile	Обновление профиля
GET	/api/v1/measurement_types	Список доступных показателей
GET	/api/v1/measurements/{type_id}	История измерений по типу
POST	/api/v1/measurements	Добавить новое измерение
DELETE	/api/v1/measurements/{id}	Удалить измерение
GET	/api/v1/goals	Список целей пользователя
POST	/api/v1/goals	Создать цель
PATCH	/api/v1/goals/{id}/complete	Отметить цель как выполненную

DELETE	/api/v1/goals/{id}	Удалить цель
GET	/api/v1/dashboard	Данные для главной страницы

Таблица 2.2 – Основные эндпоинты API